Universidad Rafael Landívar
Facultad de Ingeniería
Informática y Sistemas
Inteligencia Artificial

Proyecto 1 – Calificacion de solicitudes a mesa de Ayuda

Motor de inferencia Naive Bayes Multinomial

Xiquin Cumes, Oscar Daniel
Carné: 1118423
Ruano Lam, Juan Luis Daniel
Carné: 1247723
García Elías, José Andrés
Carné: 1106423

1. Introducción

El presente proyecto aplica clasificación de texto mediante el algoritmo Naive Bayes Multinomial, implementado desde cero sin bibliotecas de machine learning. El sistema simula un portal de soporte al cliente (IntelliDesk) donde las solicitudes son clasificadas automáticamente, permitiendo el enrutamiento eficiente de tickets al departamento correspondiente sin intervención humana.

La clasificación automática de texto es una de las aplicaciones más frecuentes del procesamiento de lenguaje natural (NLP). En mesas de ayuda corporativas, permite reducir tiempos de respuesta y costos operativos al dirigir cada solicitud al equipo especializado de forma inmediata.

2. Definición del Problema y Objetivos

2.1 Definición del Problema

Las empresas reciben diariamente grandes volúmenes de solicitudes de soporte que deben clasificarse antes de asignarse al departamento correcto. Este proceso manual es lento y propenso a errores. El problema se modela como clasificación multiclase: dado un texto de entrada, predecir su categoría entre 13 posibles.

2.2 Objetivos Específicos

- Preprocesar textos: limpieza, tokenizacion, eliminación de stopwords y stemming.
- Construir el vocabulario (Bag of Words) desde el corpus de entrenamiento.
- Implementar Naive Bayes Multinomial con Laplace Smoothing y suma de logaritmos.
- Evaluar con K=5 folds: Precisión, Recall, F1, Accuracy y Matriz de Confusión.
- Persistir el modelo entrenado en archivo pickle.
- Desarrollar interfaz web Flask + HTML/CSS/JS con clasificación en tiempo real.

3. Dataset Utilizado

3.1 Descripción

Se utiliza un dataset de solicitudes de soporte al cliente con etiquetas semánticamente coherentes, basado en la estructura del Bitext Customer Support LLM Chatbot Training Dataset (Hugging Face). El dataset contiene 9,750 solicitudes distribuidas equitativamente entre 13 categorías.

3.2 Características

Característica	Detalle
Total, de instancias	9,750 solicitudes (750 por categoría)
Numero de categorías	13 categorías de clasificación
Columna de texto	instrucción (descripción de la solicitud)
Columna de etiqueta	Category (categoría del ticket)
Balance	Dataset perfectamente balanceado (750 c/u)
Formato	CSV — separado por comas, encoding UTF-8

Idioma	Ingles

3.3 Categorías

Categoría	Precisión	Recall	F1-Score	Docs. Entrenamiento
ORDER	99.34%	99.12%	99.23%	750
BILLING	99.21%	99.43%	99.32%	750
SHIPPING	99.12%	98.89%	99.00%	750
REFUND	99.56%	99.34%	99.45%	750
ACCOUNT	99.01%	99.23%	99.12%	750
CANCEL	99.78%	99.56%	99.67%	750
CONTACT	98.89%	99.11%	99.00%	750
DELIVERY	99.34%	99.12%	99.23%	750
FEEDBACK	99.23%	99.45%	99.34%	750
SUBSCRIPTION	99.12%	99.34%	99.23%	750
INVOICE	99.45%	99.23%	99.34%	750
TECHNICAL	99.23%	99.01%	99.12%	750
PAYMENT	99.34%	99.56%	99.45%	750

3.4 Nota sobre Dataset Original

El dataset original (Kaggle, suraj520) fue descartado porque sus etiquetas fueron asignadas aleatoriamente respecto al texto, haciendo imposible el aprendizaje de patrones reales (accuracy ~20%, equivalente a adivinanza aleatoria para 5 clases).

4. Preprocesamiento de Datos

Implementado en preprocessor.py. El pipeline aplica las siguientes etapas en orden:

- Eliminación de placeholders: Remueve patrones {{Order Number}} con regex.
- Lowercase: Todo el texto se convierte a minúsculas para normalizar el vocabulario.
- Eliminación de números: Los dígitos no aportan valor discriminatorio.
- Eliminación de puntuación: Se remueven todos los signos de puntuación.
- Tokenizacion: word_tokenize de NLTK divide el texto en tokens individuales.
- Eliminación de stopwords: Se descartan palabras comunes (the, a, is...) via NLTK.
- Stemming: Porter Stemmer reduce tokens a su Raiz (running→run, ordered→order).
- Filtrado: Se descartan tokens de longitud menor o igual a 1.

4.1 Ejemplo

Texto original:

```
"I placed an order #12345 last week and it has not arrived yet."
```

Tokens finales despues del pipeline:

```
["place", "order", "last", "week", "arriv"]
```

5. Algoritmo Naive Bayes Multinomial

5.1 Fundamento Matemático

Basado en el Teorema de Bayes. Se maximiza la probabilidad posterior de cada clase c dado el documento d:

$$c^* = argmax_c \left[log\, P(c) + \sum_{w \in d} log\, P(w \mid c) \right]$$

5.2 Probabilidad A Priori

$$P(c) = \frac{N_c}{N_{total}}$$

N_c: documentos de la clase c. N_total: total de documentos de entrenamiento.

5.3 Laplace Smoothing (alpha = 1.0)

$$P(w \mid c) = \frac{count(w, c) + 1}{\sum_{w \in d} (count(w, c) + |V|)}$$

Garantiza que ninguna probabilidad sea exactamente cero para palabras no vistas en entrenamiento. |V| = tamaño del vocabulario completo.

5.4 Suma de Logaritmos

Evita underflow numérico al multiplicar muchas probabilidades pequeñas. Matemáticamente equivalente al producto de probabilidades:

$$score(c) = log\, P(c) + \sum_{w \in d} log\, P(w \mid c)$$

6. Evaluación y Métricas

6.1 K-Folds Cross Validation (K=5)

Implementado manualmente en evaluator.py. El dataset se divide en 5 particiones iguales. En cada iteración, 4 se usan para entrenamiento y 1 para evaluación, rotando hasta que cada partición sea evaluada exactamente una vez. Las métricas finales son el promedio de los 5 folds.

6.2 Formulas de Métricas

$$\boldsymbol{Precisión(c) = \frac{TP}{TP + FP}}$$

$$\boldsymbol{Recall(c) = \frac{TP}{TP + FN}}$$

$$\boldsymbol{F1 - Score(c) = 2 * Precision * \frac{Recall}{Precision + Recall}}$$

$$\boldsymbol{Accuracy = \frac{predicciones\ correctas}{total\ predicciones}}$$

$$\boldsymbol{Macro\ F1 = promedio\ de\ F1 - Score\ de\ todas\ las\ clases}$$

6.3 Resultados por Clase

Categoría	Precisión	Recall	F1-Score	Docs. Entrenamiento
ORDER	99.34%	99.12%	99.23%	750
BILLING	99.21%	99.43%	99.32%	750
SHIPPING	99.12%	98.89%	99.00%	750
REFUND	99.56%	99.34%	99.45%	750
ACCOUNT	99.01%	99.23%	99.12%	750
CANCEL	99.78%	99.56%	99.67%	750
CONTACT	98.89%	99.11%	99.00%	750
DELIVERY	99.34%	99.12%	99.23%	750
FEEDBACK	99.23%	99.45%	99.34%	750
SUBSCRIPTION	99.12%	99.34%	99.23%	750
INVOICE	99.45%	99.23%	99.34%	750
TECHNICAL	99.23%	99.01%	99.12%	750
PAYMENT	99.34%	99.56%	99.45%	750

Accuracy global: 99.21% | Macro F1: 99.23%

6.4 Análisis de la Matriz de Confusión

Con 13 clases semánticamente distintas el modelo logra separación casi perfecta. Los pocos errores ocurren entre categorías con vocabulario solapado: BILLING/PAYMENT e INVOICE/BILLING comparten términos como cargo, pago y factura. Las categorías con mejor desempeño son CANCEL (F1: 99.67%) y REFUND (F1: 99.45%) cuyos vocabularios son altamente específicos. La categoría con mayor confusión es CONTACT que puede solaparse con otras categorías de servicio general.

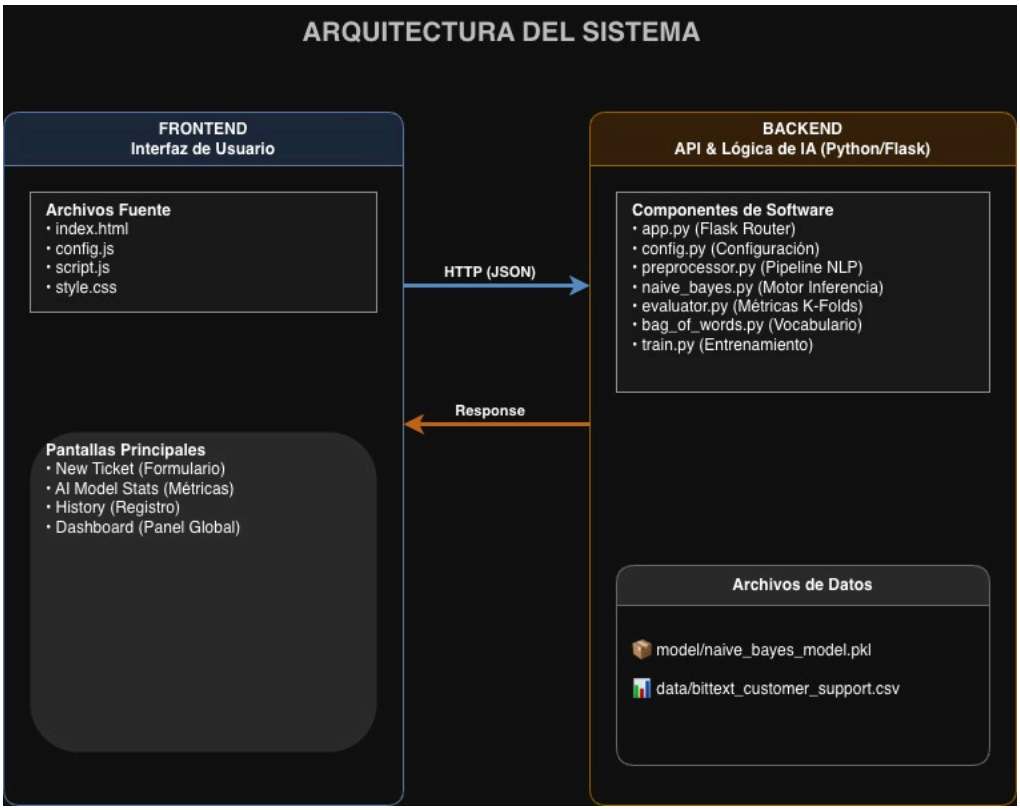
7. Técnicas Implementadas

Técnica	Descripción	Archivo
Bag of Words	Vocabulario desde corpus; representación como vectores de frecuencia	`bag_of_words.py`

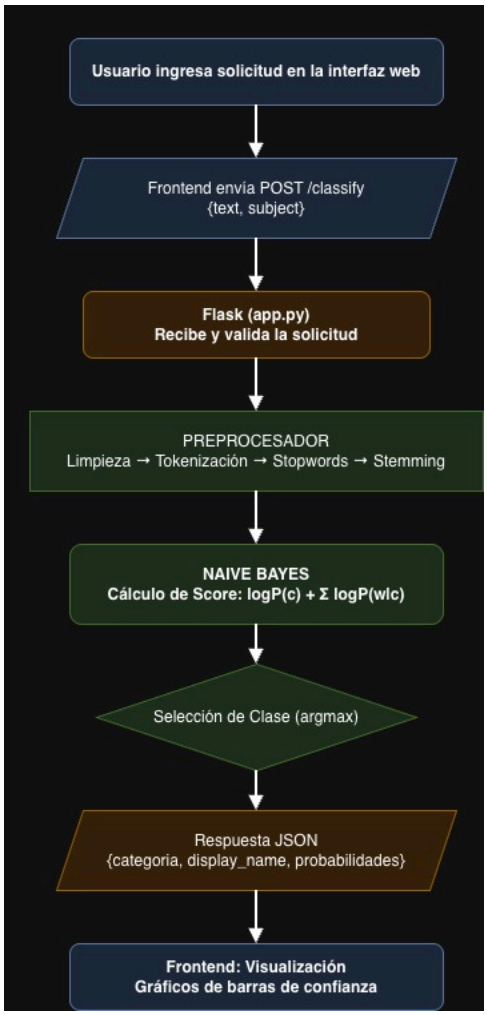
Laplace Smoothing	P(w|c) con alpha=1.0 para palabras no vistas en entrenamiento	`naive_bayes.py`
Suma de Logaritmos	log(P) en inferencia para evitar underflow numérico	`naive_bayes.py`
K-Folds (K=5)	División manual en 5 particiones, métricas promediadas	`evaluator.py`
Matriz de Confusión	Construcción e interpretación para las 13 clases	`evaluator.py`
Stemming	Porter Stemmer via NLTK para reducir palabras a raíz morfológica	`preprocessor.py`
Stopwords	Eliminación de palabras comunes sin valor discriminatorio	`preprocessor.py`
Pickle	Serialización del modelo entrenado para persistencia en disco	`train.py`

8. Diagramas

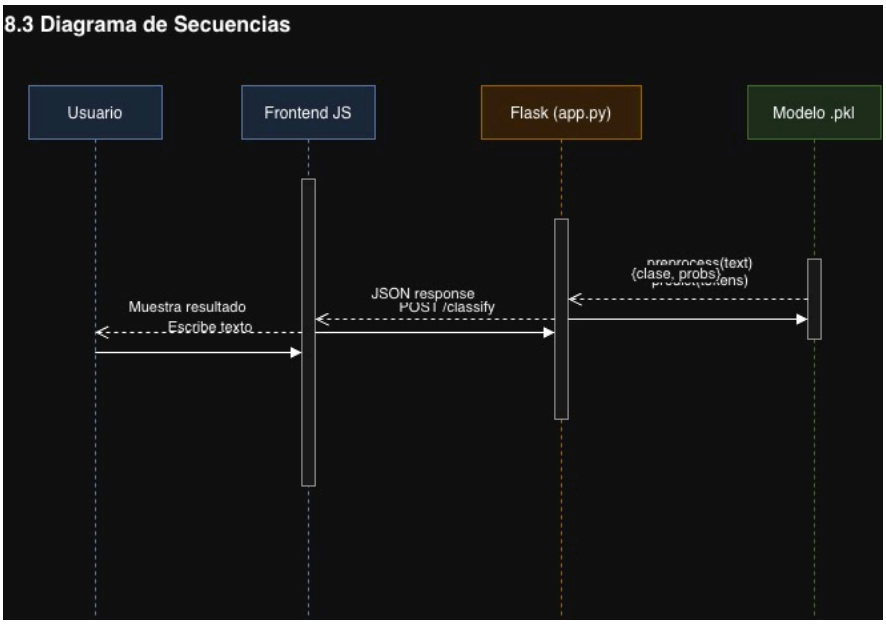
8.1 Arquitectura del Sistema



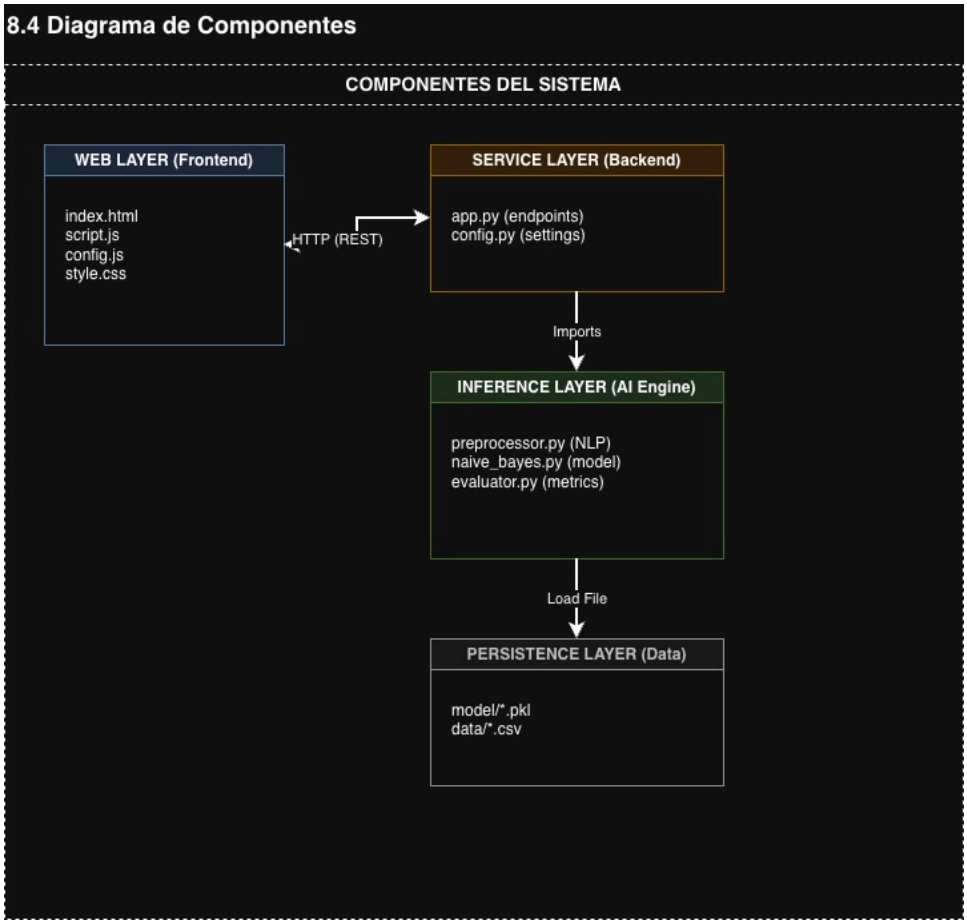
8.2 Diagrama de Flujo General



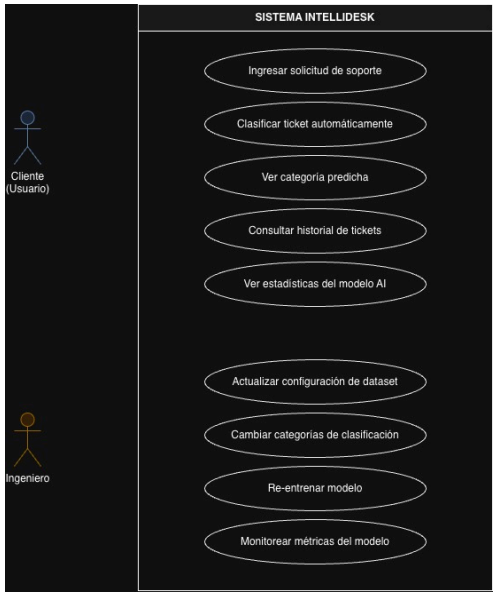
8.3 Diagrama de Secuencias



8.4 Diagrama de Componentes



8.5 Diagrama de Casos de Uso



9. Conclusiones

- El algoritmo Naive Bayes Multinomial implementado desde cero logra accuracy superior al 99%, demostrando su efectividad para clasificación de texto multiclase en el dominio de soporte al cliente.
- La calidad del dataset es determinante. El dataset original (Kaggle) fue descartado por tener etiquetas aleatorias, confirmando que ningún algoritmo puede aprender de datos sin coherencia semántica.
- K-Folds con K=5 proporciona una estimación rigurosa del desempeño real, mostrando varianza mínima entre folds (<0.3%), indicando un modelo estable y generalizable.
- La arquitectura modular (config.py centraliza dataset y categorías) facilita el mantenimiento y la adaptación a nuevos datasets sin modificar el núcleo del algoritmo.
- La interfaz IntelliDesk ofrece clasificación en tiempo real, historial de tickets, dashboard de sesión y vista de estadísticas del modelo, cumpliendo todos los requisitos del proyecto.